DESENVOLVIMENTO FRONT-END · MARCO 3

VueNautas

Gerenciador de tarefas e projetos com Vue.js e TypeScript

Thiago Henrique

Eduarda

Corina



Objetivo do Marco 3

Ampliar o cadastro e a edição de tarefas e projetos, conectar os formulários à API e melhorar validação e acessibilidade.

62

API de escrita

POST e PATCH tipados para tarefas e projetos

63–65

Tarefas

Formulário reutilizável, tela de nova tarefa e de edição

66–67

Qualidade

Validação dos campos e acessibilidade dos formulários

68

Projetos

Cadastro e edição de projeto no mesmo componente

ISSUE 69 Apresentação do Marco 3: implementação, decisões de código e motivo de cada decisão.

ISSUE 62 · API DE ESCRITA E TIPOS

Da interface aos dados

Formulários Vue
FormularioTarefa.vue
FormularioProjeto.vue

→

Módulo da API
src/api.ts
requisitar<T>()

→

API local (dev)
json-server
porta 3000

→

Armazenamento
db.json

src/tipos.ts · contrato dos dados

```
type NovaTarefa = Omit<Tarefa, "id">
type AtualizarTarefa = Partial<Omit<Tarefa, "id">>
type NovoProjeto = Omit<Projeto, "id">
type AtualizarProjeto = Partial<Omit<Projeto, "id">>
```

Por quê: a criação recebe os campos do registro; a edição aceita apenas os campos que precisam mudar.

src/api.ts · operações de escrita

Função	HTTP	Rota
criarTarefa	POST	/tarefas
atualizarTarefa	PATCH	/tarefas/:id
criarProjeto	POST	/projetos
atualizarProjeto	PATCH	/projetos/:id

Por quê: centraliza o acesso HTTP e mantém funções e payloads tipados.

ISSUES 63 · 64 · 65 · FORMULARIOTAREFA.VUE

Criar e editar tarefas

SRC/FORMULARIOTAREFA.VUE · MODO E CARREGAMENTO

```
const modoEdicao = computed(() => Boolean(rota.params.id))

watch(
  [idTarefa, () => String(rota.query.projetoId ?? '')],
  carregarFormulario,
  { immediate: true },
)
```

O mesmo componente identifica a rota e recarrega os dados quando o ID ou o projeto da URL muda.

SRC/FORMULARIOTAREFA.VUE · SALVAR E NAVEGAR

```
await atualizarTarefa(idTarefa.value, dadosTarefa)
await router.push({
  name: 'detalhe-tarefa',
  params: { id: idTarefa.value },
})

// criação
await criarTarefa(dadosTarefa)
await router.push('/')
```

POST retorna à lista; PATCH abre o detalhe da tarefa editada.

Um formulário, dois modos

computed deriva cadastro ou edição a partir do parâmetro :id.

Dados carregados pela API

watch coordena a busca; v-model preenche os campos com os dados retornados.

Projeto pré-selecionado

?projetoId= permite abrir a criação já vinculada ao projeto de origem.

Salvar e navegar

POST volta à lista; PATCH volta aos detalhes da tarefa.

Falha ao carregar

erroCarregamento separa o erro de busca do erro de salvamento e oferece retorno.

Validação e acessibilidade

FORMULARIOTAREFA.VUE · ENVIO INVÁLIDO

```
if (!validarFormulario()) {
  await nextTick()
  formularioRef.value
    ?.querySelector('[aria-invalid="true"]')
    ?.focus()
  return
}
```

Impede o envio e direciona o foco ao primeiro erro.

FORMULARIOPROJETO.VUE · CAMPO E MENSAGEM

```
:aria-invalid="Boolean(erros.nome)"
:aria-describedby="erros.nome
  ? 'projeto-campos-obrigatorios erro-nome'
  : 'projeto-campos-obrigatorios'"
<small v-if="erros.nome" role="alert">
```

O campo é ligado à explicação de obrigatoriedade e, quando há erro, à mensagem específica.

Validação · FormularioTarefa.vue

5 campos obrigatórios. validarFormulario() barra dados incompletos; validarCampo() reavalia ao digitar, sair do campo ou selecionar.

Acessibilidade · os dois formulários

label associado por for/id; mensagens em texto; aria-invalid e aria-describedby; role="alert"; foco no primeiro campo inválido; controles bloqueados enquanto salva.

Projeto · FormularioProjeto.vue

validarFormulario() confere nome, descrição e disciplina; validarCampo() remove a mensagem quando o valor fica válido. O estado erroCarregamento evita mostrar um formulário vazio quando a busca falha.

ISSUE 68 · FORMULARIOPROJETO.VUE

Cadastro e edição de projetos

SRC/FORMULARIOPROJETO.VUE · CARREGAR OS DADOS

```
watch([idProjeto, modoEdicao], carregarFormulario, { immediate: true })

const numeroRequisicao = ++requisicaoAtual
const projeto = await buscarProjeto(idProjeto.value)
if (numeroRequisicao !== requisicaoAtual) return
```

Recarrega pela rota e ignora respostas antigas de uma requisição anterior.

SRC/FORMULARIOPROJETO.VUE · GRAVAÇÃO

```
if (modoEdicao.value) {
  await atualizarProjeto(idProjeto.value, dadosProjeto)
  await router.push({
    name: 'detalhe-projeto',
    params: { id: idProjeto.value },
  })
} else {
  await criarProjeto(dadosProjeto)
  await router.push('/projetos')
}
```

POST volta à lista; PATCH retorna aos detalhes do projeto editado.

Duas rotas, um componente

/projetos/novo e /projetos/:id/editar; computed define o modo a partir do ID.

Carregamento com controle de erro

watch chama buscarProjeto; erroCarregamento impede exibir campos vazios como se estivessem carregados.

criadoEm preservado

Cadastro gera a data; edição mantém a data original.

Projeto com suas tarefas

src/api.ts busca /projetos/:id?_embed=tarefas; DetalheProjeto.vue reaproveita CartaoTarefa.

BALANÇO DO MARCO 3

O que evoluiu no código

Escrita de dados pela API

src/api.ts · src/tipos.ts

POST e PATCH tipados; tipos distinguem criação e atualização.
Por quê: a interface envia dados com contrato definido.

Criação e edição de tarefas

src/FormularioTarefa.vue · src/router.ts

Um componente serve às rotas de nova tarefa e edição.
Por quê: reaproveita lógica e mantém o fluxo coerente.

Cadastro e edição de projetos

src/FormularioProjeto.vue · src/DetalheProjeto.vue

watch carrega os dados; criadoEm é preservado; detalhe mostra tarefas vinculadas.
Por quê: manter contexto e evitar dados desatualizados.

Validação e acessibilidade

FormularioTarefa.vue · FormularioProjeto.vue

Erros por campo, atributos ARIA, foco no primeiro inválido.
Por quê: impedir envio incompleto e orientar todos os usuários.

O sistema deixou de só consultar dados e passou a criar, atualizar e orientar o usuário durante o preenchimento.

CÓDIGO `github.com/thiagopietrobon/por-enquanto-nao-sabemos`

Antes de fechar as issues: executar npm run build e testar teclado, cadastro/edição e falha da API.